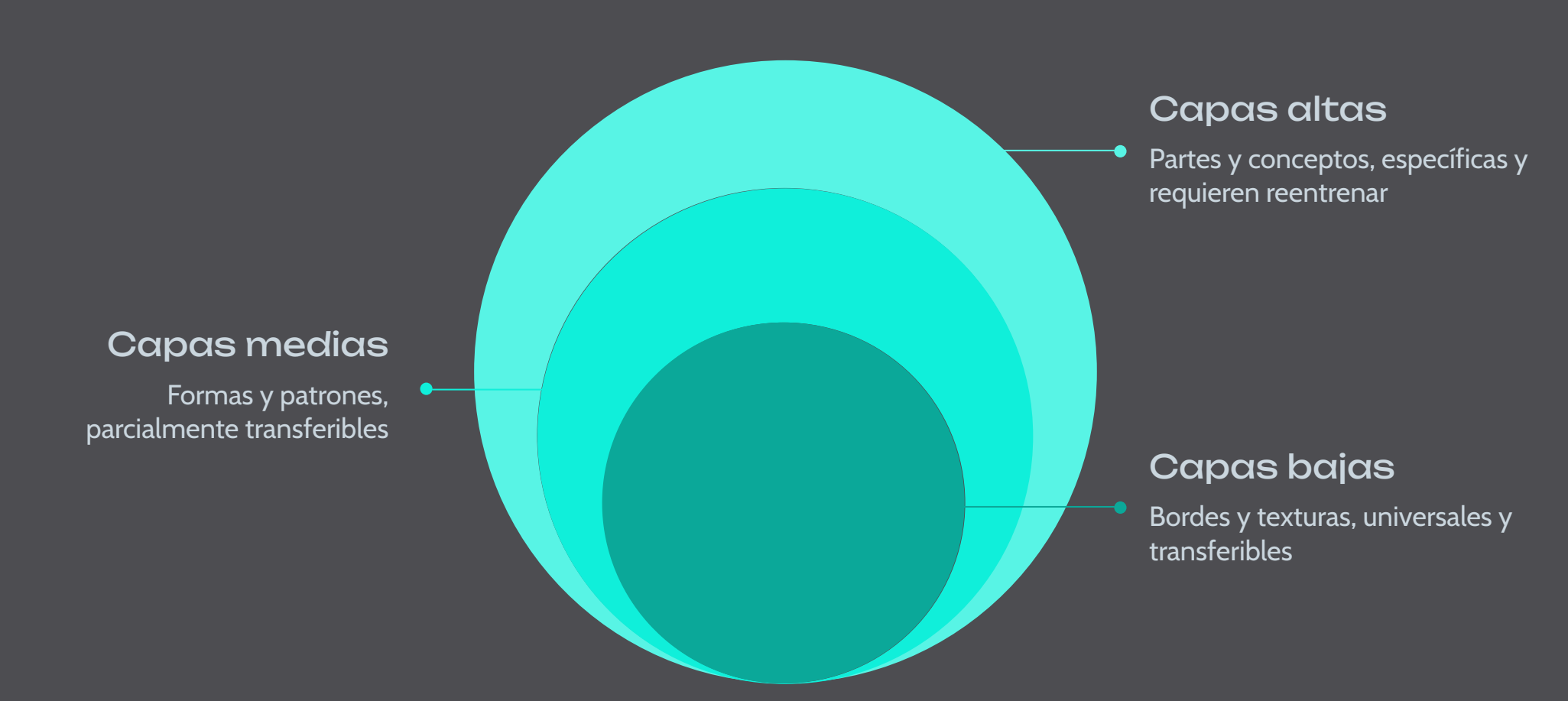


Transfer Learning: Fundamentos y Matriz de Decisión

El **Transfer Learning** reutiliza pesos de modelos preentrenados (ImageNet) para nuevas tareas con pocos datos. Las primeras capas aprenden características universales (bordes, texturas) transferibles a cualquier problema visual; las últimas capturan conceptos específicos del dataset original. La clave está en decidir qué capas congelar y cuáles reentrenar según el tamaño de tu dataset y su similitud con el dominio original.

Memoria Jerárquica de una CNN

Una CNN construye representaciones de forma incremental: las primeras capas detectan patrones simples y universales, mientras que las profundas combinan esos patrones en conceptos abstractos y específicos del dominio.



Las capas bajas son casi universales y se congelan siempre; las medias se ajustan según la similitud del dominio; las altas se reentrenan cuando el dataset es grande o el dominio es diferente.

Matriz de Decisión 2×2

La elección entre **Feature Extraction** y **Fine-Tuning** depende de dos factores: el **tamaño del dataset** y su **similitud con el dominio original**. Identifica tu caso y aplica la estrategia correspondiente.

	Dataset Pequeño	Dataset Grande
Similar al dominio	Feature Extraction: Congelar base completa + clasificador propio. Riesgo mínimo de overfitting, entrenamiento rápido.	Fine-Tuning: Feature Extraction inicial + descongelar capas altas (2-3 últimas) + reentrenar con lr baja.
Diferente del dominio	Feature Extraction: Congelar solo capas bajas (1-2 bloques). Las medias/altas capturan conceptos no transferibles.	Fine-Tuning Agresivo: Feature Extraction inicial + descongelar más capas (4-5 bloques) + reentrenar con lr muy baja.

Feature Extraction vs Fine-Tuning: Comparación Técnica

Feature Extraction es el punto de partida obligatorio: la base congelada actúa como extractor de características fijo. Fine-Tuning es una optimización posterior que solo se aplica tras validar el rendimiento inicial.

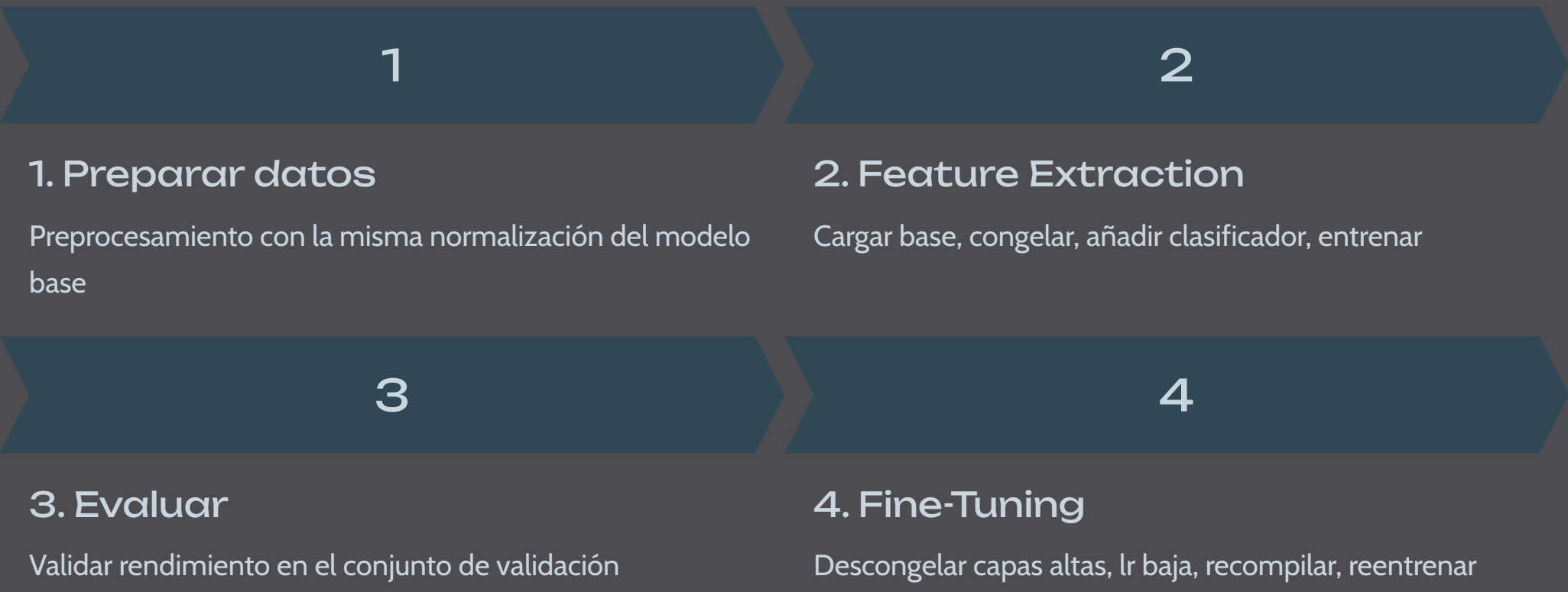
Feature Extraction

- Base preentrenada **completamente congelada** (trainable=False)
- Solo se entrena el clasificador añadido
- Rápido, eficiente, ideal para datasets pequeños
- Riesgo de underfitting si el dominio es muy diferente
- Siempre es el primer paso**

Fine-Tuning

- Se aplica **después** de entrenar el clasificador
- Descongelar capas altas de la base (2-3 últimas)
- Reentrenar con **learning rate muy bajo** (1e-5 a 1e-4)
- Mejora la adaptación al dominio específico
- Requiere **recompilar** el modelo tras descongelar

Flujo de Trabajo Recomendado



Implementación en Keras: Código y Buenas Prácticas

Esta sección contiene los tres bloques de código esenciales para implementar Transfer Learning en Keras: cargar y congelar la base, añadir el clasificador personalizado y ejecutar el Fine-Tuning correctamente.

Bloque 1 — Cargar y Congelar la Base Preentrenada

```
# =====
# BLOQUE 1: Cargar y Congelar la Base
# =====

import tensorflow as tf
from tensorflow.keras.applications import VGG16

# Cargar base preentrenada sin la cabeza clasificadora original
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

# Congelar TODAS las capas de la base (Feature Extraction)
base_model.trainable = False

# Verificar cantidad de parámetros
base_model.summary()
```

Bloque 2 — Añadir Clasificador Personalizado y Entrenar

```
# =====
# BLOQUE 2: Añadir Clasificador y Entrenar
# =====

from tensorflow.keras import layers, models

# Construir la arquitectura completa usando Functional API
inputs = tf.keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False) # Mantener BatchNorm en modo inferencia
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.5)(x)             # Regularización contra overfitting
outputs = layers.Dense(num_clases, activation='softmax')(x)

model = models.Model(inputs, outputs)

# Compilar el modelo (solo la cabeza es entrenable)
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Entrenar la baseline (Feature Extraction)
history = model.fit(
    train_generator,
    epochs=10,
    validation_data=val_generator
)
```

Bloque 3 — Fine-Tuning: Descongelar y Reentrenar

```
# =====
# BLOQUE 3: Fine-Tuning (Descongelar y Reentrenar)
# =====

# Descongelar las últimas N capas de la base (ejemplo: últimas 4)
N_capas_a_ajustar = 4

base_model.trainable = True
for layer in base_model.layers[-N_capas_a_ajustar]:
    layer.trainable = False

# RECOMPILACIÓN OBLIGATORIA con un Learning Rate bajo
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Reentrenar el modelo adaptado
history_finetune = model.fit(
    train_generator,
    epochs=5,
    validation_data=val_generator
)
```